

Working with the Shape-Drawing Functions

The `drawFractal()` function uses a shape-drawing function passed to it to draw the individual parts of the fractal. This is usually a simple shape, such as a square or triangle. The beautiful complexity of the fractals emerges from `drawFractal()` recursively calling this function for each individual component of the whole fractal.

The shape-drawing functions for the Fractal Art Maker have two parameters: `size` and `depth`. The `size` parameter is the length of the sides of the square or triangle it draws. The shape-drawing functions should always use arguments to `turtle.forward()` that are based on `size` so that the lengths will be proportionate to `size` at each level of recursion. Avoid code like `turtle.forward(100)` OR `turtle.forward(200)`; instead, use code that is based on the `size` parameter, like `turtle.forward(size)` OR `turtle.forward(size * 2)`. In Python's `turtle` module, `turtle.forward(1)` moves the turtle by one *unit*, which is not necessarily the same as one pixel.

The shape-drawing functions' second parameter is the recursive depth of `drawFractal()`. The original call to `drawFractal()` has the `depth` parameter set to 0. Recursive calls to `drawFractal()` use `depth + 1` as the `depth` parameter. In the Wave fractal, the first triangle in the center of the window has a `depth` argument of 0. The three triangles created next have a `depth` of 1. The nine triangles around those three triangles have a `depth` of 2, and so on.

Your shape-drawing function can ignore this argument, but using it can cause interesting variations to the basic shape. For example, the `drawFilledSquare()` shape-drawing function uses `depth` to alternate between drawing white squares and gray squares. Keep this in mind if you'd like to create your own shape-drawing functions for the Fractal Art Maker program, as they must accept a `size` and `depth` argument.

The drawFilledSquare() Function

The `drawFilledSquare()` function draws a filled-in square with sides of length `size`. To color the square, we use the `turtle` module's `turtle.begin_fill()` and `turtle.end_fill()` functions to make the square either white or gray, with a black border, depending on whether the `depth` argument is even or odd.